Suchen auf Arrays

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B

Version: 1.1.8.1

Quelle: Die Folien sind teilweise inspiriert von oder basierend auf Lehrmaterial von Prof. Dr. Ritterbusch.

Kontrollfragen: kontrollfragen.de.md.html
Folien: https://delors.github.io/theo-algo-suchen_auf_arrays/folien.de.md.html
https://delors.github.io/theo-algo-suchen_auf_arrays/folien.de.md.html.pdf
Fehler melden: https://github.com/Delors/delors.github.io/issues
KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Einführung

Skalierung von Daten

Welche Skalierung haben gesuchte Daten sind im Array?

nominal: Nur Vergleich auf Gleichheit, keine natürliche Ordnung oder Zahlbegriff.

Ein Beispiel für eine *nominal* skalierte Datenmenge wäre die Menge der Farben. Es gibt keine natürliche Ordnung der Farben, und es gibt auch keinen natürlichen Zahlenbegriff, der die Farben beschreibt. Ein weiteres Beispiel ist eine Liste von Wohnorten.

ordinal: Es gibt Größenvergleiche und damit eine Sortierung, aber kein Zahlbegriff.

Ein Beispiel für eine *ordinale* skalierte Datenmenge wäre die Menge der Kleidergrößen (S,M,L,XL,...). Es gibt eine natürliche Ordnung der Kleidergrößen, aber es gibt keinen natürlichen Zahlenbegriff, der die Kleidergrößen beschreibt. Ein weiteres Beispiel ist die Bewertung von Filmen auf einer Skala von 1 bis 5 Sternen.

kardinal: Es gibt Größenvergleiche und Zahlbegriff.

Beobachtung

- **Unsortiert oder nominal** führt (zunächst) zur *linearen Suche*.
- **Ordinale und kardinale Werte** können *sortiert werden für binäre Suche*.
- **Kardinale Größen** können *modelliert werden für interpolierende Suche*.

Hinweis

Für unsere Betrachtung gehen wir im Folgenden davon aus, dass die Daten sortiert sind. Beim Vergleich der Algorithmen beschränken wir uns auf eine Betrachtung der Anzahl der Elementzugriffe.

Lineare Suche

```
// A        1-indiziertes Array, das durchsucht wird
// n        Größe des Arrays
// needle   der zu suchende Wert
function LinearSearch(A,n,needle)
    for i := 1,...,n do
        if A[i] == needle then
            return i
    return nil
```

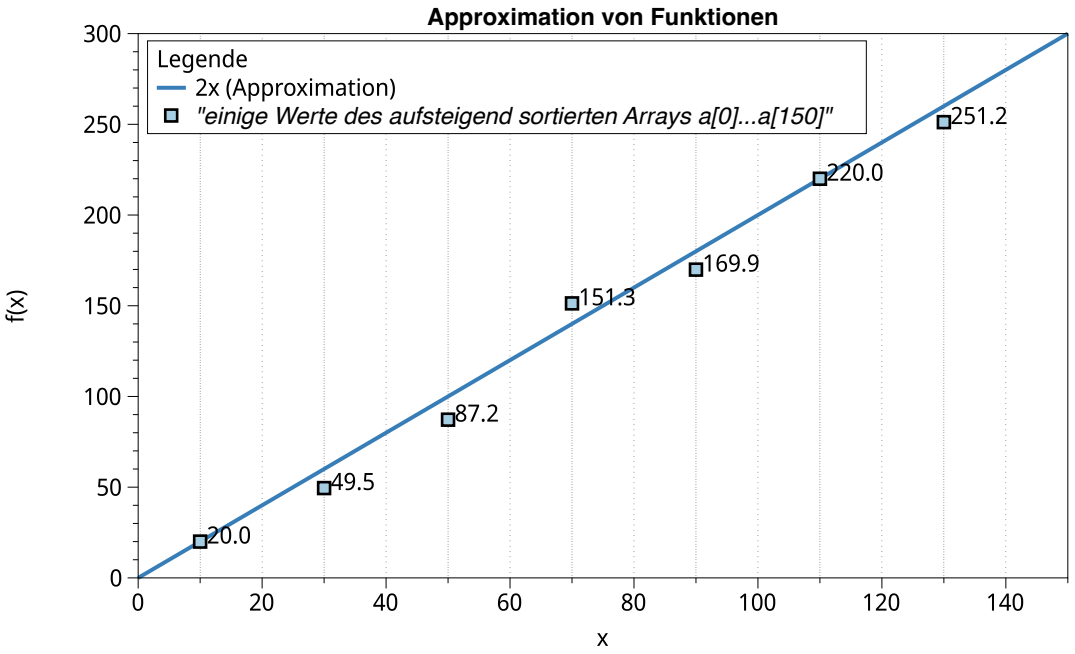
Laufzeit und Elementzugriffe kann asymptotisch durch $O(n)$ abgeschätzt werden.

Binäre Suche

```
1  // A        aufsteigend sortiertes Array, das durchsucht wird
2  // l        untere Grenze (Index im Array)
3  // u        obere Grenze  (Index im Array: untere Grenze ⩽ obere Grenze)
4  // needle   der zu suchende Wert
5  function BinarySearch(A,l,u,needle)
6      upper := u
7      lower := l
8      repeat
9          pos     := floor((upper + lower) / 2)
10         value   := A[pos]
11         if      value == needle then return pos
12         else if value > needle  then
13                 upper := pos - 1
14         else    lower := pos + 1
15     until upper < lower
16     return nil
```

Laufzeit ist $O(\log(n))$, genauer im Schnitt $\log_2(n) - 1$ Zugriffe.

Effizientere Suche bei linearer Verteilung



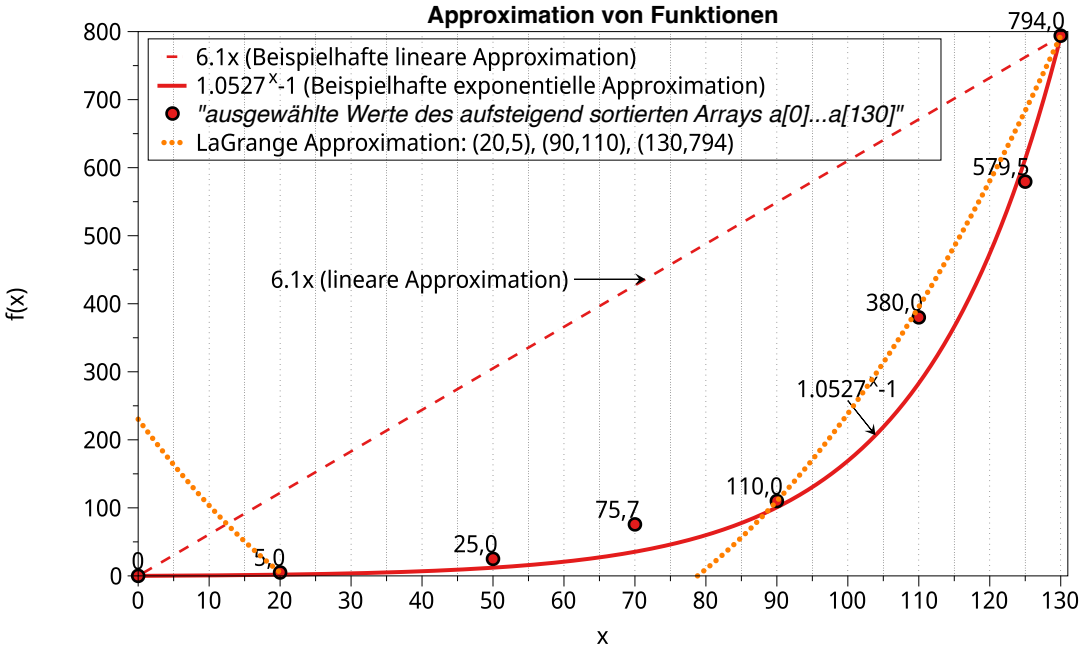
In diesem Beispiel gehen wir davon aus, dass die Werte *im Wesentlichen* linear verteilt sind. Das bedeutet, dass die Differenz zwischen zwei aufeinanderfolgenden Werten immer gleich ist.

Sei beispielsweise ein Array `a` mit folgenden Werten geben (Auszug):

Index i	Wert
i = 10	a[i = 10] = 20.0
...	...
i = 30	49.5
...	...
i = 50	87.2
...	...
i = 70	151.3
...	...
i = 90	169.9
...	...
i = 110	220.0
...	...
i = 130	251.2

Wenn man jetzt exemplarisch die Paare: $(i = 10, a[i] = 20.0)$, und $(i = 110, a[i] = 220)$ betrachtet, dann kann man zu dem Schluss kommen, dass die Funktion $f(x) = 2.0 \cdot x$ eine Approximation der Verteilung der Werte ist. Würde man also nach dem Wert $a[i] = y = 170$ suchen wollen, dann wäre es gut als Erstes den Wert von $a[85]$ zu überprüfen, $170 = 2 \cdot x \rightarrow \frac{170}{2} = 85 = i$.

Effizientere Suche bei exponentieller Verteilung



In diesem Beispiel gehen wir davon aus, dass die Werte *im Wesentlichen* exponentiell verteilt sind. Das bedeutet, dass die Differenz zwischen zwei aufeinanderfolgenden Werten immer größer wird.

Sei beispielsweise ein Array `a` mit folgenden Werten geben (Auszug):

index i	a[i]
0	0
...	...
20	5
...	...
50	25
...	...
70	75.7
...	...
90	110
...	...
110	380
...	...
125	579.5
...	...
130	794

Wenn man jetzt exemplarisch die Paare: $(i = 20, a[i] = 5.0)$, und $(i = 130, a[i] = 794)$ betrachtet, und eine lineare Approximation durchführt, dann könnte man zu dem Schluss kommen, dass die Funktion $f(x) = 6.1 \cdot x$ eine gute Approximation ist.

Würde man eine quadratische Approximation mit Hilfe von Lagrange durchführen, zum Beispiel mit den Werten $(i = 20, a[i] = 5.0)$, $(i = 90, a[i] = 110)$, und $(i = 130, a[i] = 794)$. Dann wäre der Fehler zwischen der realen Verteilung und der angenommen deutlich geringer, da die quadratische Funktion die Werte besser approximiert.

In diesem Fall wäre die Funktion: $p(x) = \frac{39}{275}x^2 - \frac{141}{10}x + \frac{2533}{11}$. In diesem Fall können wir die Position des Wertes 650 im Array besser abschätzen (durch die Aufstellung der Umkehrfunktion und dann einsetzen von 650): $\approx \mathbf{123}$.

Warnung

Eine vernünftige Interpolation ist nur dann möglich, wenn die Verteilung der Werte im Wesentlichen bekannt ist.

Approximation der Verteilung

!! Wichtig

Wenn wir die Verteilung der Werte kennen, können wir effizientere Algorithmen entwickeln.

Beispiel

Wenn wir wissen, dass die Werte quadratisch verteilt sind (`Array[10] a = { 1, 4, 9, 16, ..., 100 }`), und wir zum Beispiel wissen, dass der kleinste Wert im Array 1 und der größte Wert 100 (an Stelle/mit Index 10) ist, den wir im Array gespeichert haben, dann macht es „keinen“ Sinn den Wert 85 oder 5 in der Mitte zu suchen! (85 findet sich vermutlich an Stelle $9 = \lfloor \sqrt{85} \rfloor$).

Interpolation mit Lagrange-Polynomen

Speichert unser Array kardinal skalierte Daten, so können diese modelliert werden. Das einfachste Prinzip ist die Polynominterpolation mittels Lagrange-Polynomen.

Das Ziel ist es, ein Polynom $p(x)$ zu finden, das eine Funktion $f(x)$ an einer gegebenen Menge von Punkten $(x_1, y_1), \ldots, (x_n, y_n)$ *exakt* interpoliert. Das heißt:

$$p(x_i) = y_i \quad \text{für alle } i = 1, \ldots, n$$

⊞ Satz

Das Lagrange-Interpolationspolynom $p(x)$ wird als Summe von Lagrange-Basispolynomen $l_i(x)$ aufgebaut:

$$p(x) = \sum_{i=1}^{n} (y_i \cdot l_i(x))$$

wobei $l_i(x)$, das i-te Lagrange-Basispolynom, gegeben ist durch:

$$l_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

◆ Bemerkung

Warum ist diese Konstruktion korrekt? (Intuition)

Man kann beobachten, dass für einen gegebenen Punkt (x_i, y_i) nur das Lagrange-Basispolynom L_i an der Stelle x_i nicht zu 0 wird, da x_i kein Faktor im Produkt von L_i ist; oder anders ausgedrückt: einer der Faktoren $(x_i - x_k)$ im Produkt der anderen Basispolynome wird garantiert zu 0 , d. h. es gilt $L_j(x_i) = 0$ für alle $j \neq i$.

Das Lagrange-Basispolynom L_i für den Punkt (x_i, y_i) ist weiterhin so konstruiert, dass es zu eins evaluiert, womit die Multiplikation mit y_i notwendigerweise den gewünschten Zielwert ergibt. Alle anderen Summanden sind 0.

Sind n Tupel $(x_n, y_n) \in \mathbb{R}^2$ reeller Zahlen gegeben mit $x_l \neq x_m$ für $l \neq m$.

Das Lagrange-Interpolationspolynom hat dann höchstens den Grad $n - 1$ und es gilt $p(x_i) = y_i$ für alle $i = 1, ..., n$.

⧉ Beispiel

Gegeben sein die zwei Punkte: $(x_1, y_1) = (1, 2)$ und $(x_2, y_2) = (3, 4)$.

Das Lagrange-Polynom $p(x)$ ist dann:

1. $$l_1(x) = \frac{x - x_2}{x_1 - x_2} = \frac{x - 3}{1 - 3} = \frac{3 - x}{2}$$

2. $$l_2(x) = \frac{x - x_1}{x_2 - x_1} = \frac{x - 1}{3 - 1} = \frac{x - 1}{2}$$

3. $$p(x) = y_1 \cdot l_1(x) + y_2 \cdot l_2(x) = 2 \cdot \frac{3 - x}{2} + 4 \cdot \frac{x - 1}{2} = x + 1$$

Nach Ausmultiplizieren und Zusammenfassen ergibt das ein Polynom, das durch beide Punkte verläuft.

Wenn zwei Punkte gegeben sind, ist das Lagrange Polynom somit:

$$p(x) = y_1 \cdot \frac{x - x_2}{x_1 - x_2} + y_2 \cdot \frac{x - x_1}{x_2 - x_1}$$

Bei drei Punkten ist das Lagrange Polynom somit:

$$\begin{aligned} p(x) = \quad & y_1 \cdot \tfrac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)} + \\ & y_2 \cdot \tfrac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)} + \\ & y_3 \cdot \tfrac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)} \end{aligned}$$

Der Grad unseres Lagrange-Polynoms ist immer um 1 kleiner als die Anzahl der gegebenen Punkte (die Terme des Basispolynoms sind nur für $j \neq i$ definiert). Das bedeutet, dass wir für zwei Punkte ein lineares Polynom erhalten, für drei Punkte ein quadratisches Polynom, für vier Punkte ein kubisches Polynom, und so weiter. Weiterhin stellt die Konstruktion sicher, dass wir durch alle gegebenen Punkte gehen.

✎ Übung

1.1. Bestimme p(2)

Bestimmen Sie direkt $p(2)$ für das quadratische Polynom mit den Eigenschaften:

$$p(-10) = 3, p(-8) = 1, p(-4) = -1$$

1.2. Bestimme p(-1)

Für die gegebenen Punkte, bestimmen Sie erst das Lagrange Polynom $p(x)$ im Allgemeinen und rechnen Sie dann den Wert für $p(-1)$ aus.

$$p(2) = 4, p(4) = 6, p(7) = 3$$

Interpolierende Suche - lineare Approximation

Beispiel

Gegeben:

Vom Array a sei bekannt: `a[1] = 0`, `a[20] = 30` und `a[40] = 120`.

Frage: Ist der Wert `50` im Array enthalten?

Lösung: Das Lagrangepolynom $p(x)$ mit $p(30) = 20$ und $p(120) = 40$ lautet:

$$p(x) = 20 \cdot \frac{x - 120}{30 - 120} + 40 \cdot \frac{x - 30}{120 - 30}$$

Für den gesuchten Wert 50 ergibt sich als zu untersuchende Position:

$$p(50) = 20 \cdot \frac{50 - 120}{30 - 120} + 40 \cdot \frac{50 - 30}{120 - 30} = \frac{220}{9} \approx 24$$

Bemerkung

Wir möchten die Position des Wertes 50 im Array abschätzen! Deswegen sind im linearen Modell die Paare $(x_1, y1) = (30, 20)$ und $(x_2, y_2) = (120, 40)$ zu wählen. D. h. die Indizes sind unsere y-Werte.

Eine binäre Suche würde in diesem Fall mit der Position $\frac{40+20}{2} = 30$ beginnen.

Hinweis

Das Lagrangepolynom kann per Konstruktion die Position der Werte 30 und 120 perfekt bestimmen:

$$p(30) = 20 \cdot \frac{30 - 120}{30 - 120} + 40 \cdot \frac{30 - 30}{120 - 30} = 20$$

$$p(120) = 20 \cdot \frac{120 - 120}{30 - 120} + 40 \cdot \frac{120 - 30}{120 - 30} = 40$$

Interpolierende Suche - quadratische Approximation

Beispiel

Gegeben:

Vom Array a sei bekannt: `a[1] = 0`, `a[20] = 30` und `a[40] = 120`.

Frage: Ist der Wert 50 im Array enthalten?

Lösung: $p(x)$ mit $p(0) = 1, p(30) = 20$ und $p(120) = 40$ lautet:

$$\begin{aligned} p(x) = \quad & 1 \cdot \tfrac{(x-30)(x-120)}{(0-30)(0-120)} + \\ & 20 \cdot \tfrac{(x-0)(x-120)}{(30-0)(30-120)} + \\ & 40 \cdot \tfrac{(x-0)(x-30)}{(120-0)(120-30)} \end{aligned}$$

Für den gesuchten Wert 50 ergibt sich als zu untersuchende Position:

$$p(50) \approx 29$$

Bemerkung: Vandermonde-Ansatz

Alternativ kann man auch den Ansatz über die Monombasis (auch Vandermonde-Ansatz genannt) wählen.

Gegeben seien die Stützpunkte: $(-53, 1)$, $(-11, 31)$, $(59, 61)$, gesucht $p(42)$ mit $p(v) = av^2 + bv + c$.

Gleichungssystem (Wert einsetzen, Index als rechte Seite):

$$\begin{aligned} 2809a - 53b + c &= 1 \qquad (\mathrm{I}) \\ 121a - 11b + c &= 31 \qquad (\mathrm{II}) \\ 3481a + 59b + c &= 61 \qquad (\mathrm{III}) \end{aligned}$$

c eliminieren:

$$(\mathrm{II}) - (\mathrm{I}): \quad -2688a + 42b = 30 \implies -448a + 7b = 5$$

$$(\mathrm{III}) - (\mathrm{II}): \quad 3360a + 70b = 30 \implies 336a + 7b = 3$$

b eliminieren:

$$784a = -2 \implies a = -\frac{1}{392}$$

Rückwärts einsetzen:

$$7b = 3 - 336\left(-\tfrac{1}{392}\right) = 3 + \tfrac{6}{7} = \tfrac{27}{7} \implies b = \frac{27}{49}$$

$$c = 31 - 121a + 11b = 31 + \tfrac{121}{392} + \tfrac{297}{49} = \frac{14649}{392}$$

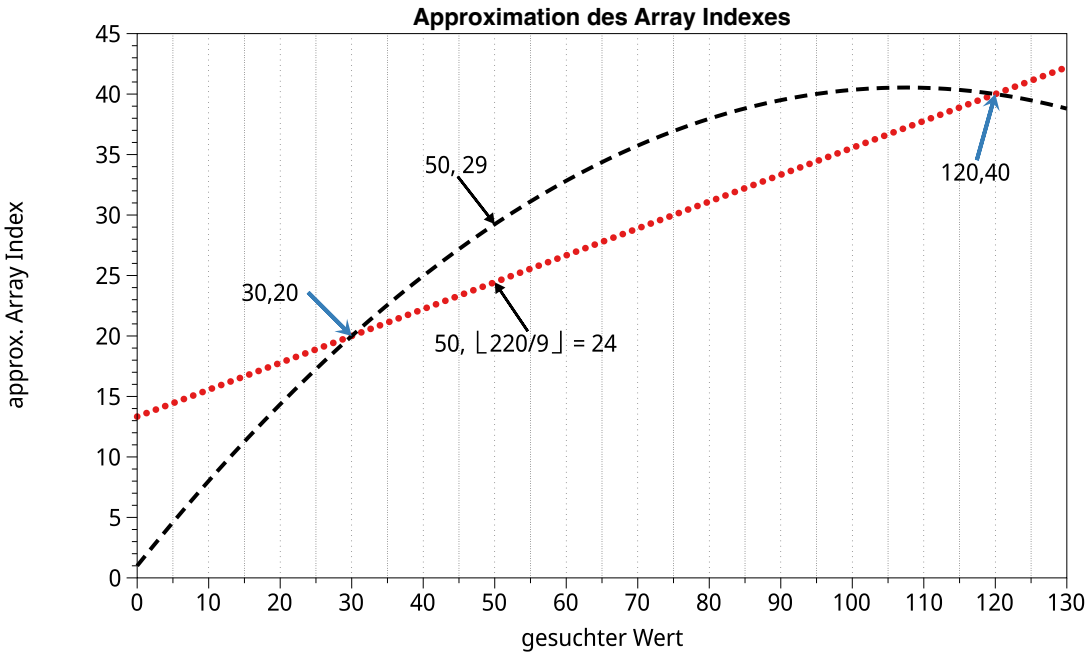
Also $p(v) = -\frac{1}{392}v^2 + \frac{27}{49}v + \frac{14649}{392}$

Probe: $p(-53) = \frac{-2809-11448+14649}{392} = 1$ ✓

▲ Achtung!

Wenn in der Klausur explizit LaGrange erwähnt wird, dann wird eine Lösung mittels des Vandermonde Ansatzes nicht akzeptiert. Sollte die Klausur nur von quadratischer Interpolation sprechen, dann sind Sie frei in der Wahl des Vorgehens.

Interpolierende Suche - Vergleich



Zusammenfassung

- Auf gleichverteilten Daten hat die lineare Interpolationssuche O(log log n).
- Auf anderen Verteilungen ist lineare Interpolation oft schlechter als binäre Suche.
- Quadratische Interpolation hat ein erweitertes Modell und schlägt binäre Suche häufig.

Lineare interpolierende Suche

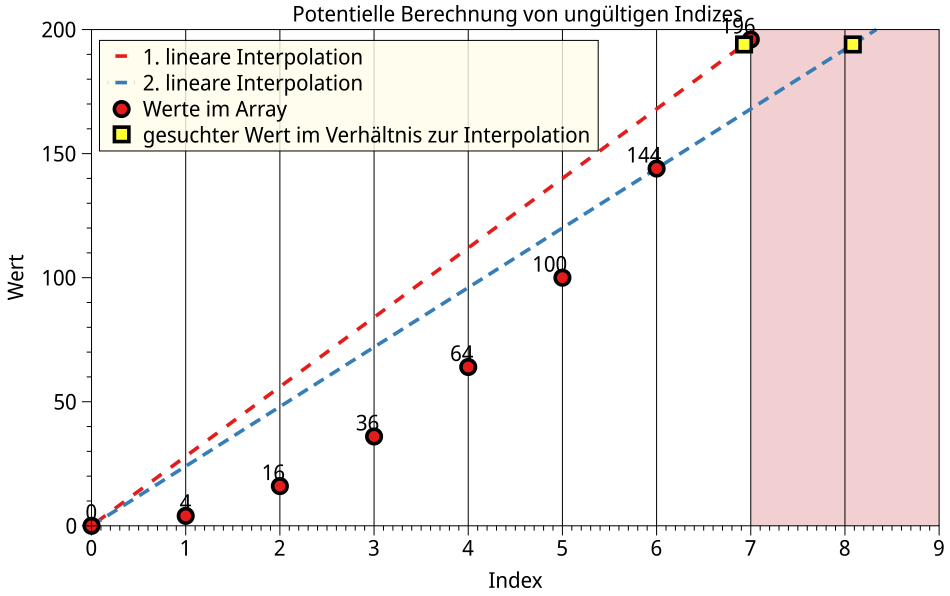
```
1  // A        durchsuchtes, 1-indiziertes, aufsteigend sortiertes Array
2  // needle   der Wert, der gesucht wird
3  function linearInterpolatingSearch(A,needle)
4      lower    := 1               // index auf das kleinste Element
5      upper    := length(A)       // index auf das größte Element
6      vL       := A[lower]
7      if vL == needle then return lower
8      vU       := A[upper]
9      if vU == needle then return upper
10     while upper > lower do
11         pos      := round(lower·(needle−vU)/(vL−vU) +
12                           upper·(needle−vL)/(vU−vL))
13         pos      := max(lower + 1, min(upper - 1, pos))
14         value    := A[pos]
15         if value == needle then return pos
16         else if value < needle then
17                  lower := max(pos, lower+1), vL := A[lower]
18         else     upper := min(pos, upper-1), vU := A[upper]
19     return nil
```

◆ Bemerkung

Die Korrektur von pos in Zeile 11 (`pos := max(lower + 1, min(upper - 1, pos))`) stellt sicher, dass pos immer zwischen `lower` und upper liegt. Dies ist insbesondere deswegen notwendig, weil die Interpolation nicht immer exakt ist. Stellen Sie sich zum Beispiel vor, dass die Daten polynomiell skaliert sind und sie (in Unkenntnis der echten Verteilung) die lineare Interpolationssuche verwenden. In diesem Fall kann es zu folgender Situation kommen:

Die Werte im Array seien: [0, 4, 16, 36, 64, 100, 144, 196] (zu Grunde liegt die Funktion $4x^2$) und Sie suchen nach dem Wert 194.

Im ersten Schritt würde die lineare Interpolationssuche den Wert 194 auf Position 7 schätzen, was nutzlos wäre, aber erst einmal kein Problem verursachen würde. Da der Wert 194 aber nicht im Array enthalten ist, würde die Suche den Wert für die obere Grenze um eins korrigieren. Jetzt würde die lineare Interpolation aber mit den Werten des Arrays an Stelle 0 und 6 erfolgen (A[0] = 0 und A[6] = 144). Das Ergebnis wäre die 2. Funktion (blau) und der Wert 194 würde auf Position 8 geschätzt, was außerhalb des Arrays liegt.



Folgen die Werte im Array einer logarithmisch Verteilung, dann würde die umgekehrte Situation eintreten, d. h. es könnte am unteren Ende des Arrays zu einem ähnlichen Problem kommen, da dann die Werte oberhalb der geraden liegen würden.

Wenn der berechnete Index außerhalb des Bereichs ist, dann kann der Algorithmus auch einfach `nil` zurückgeben, da der Wert dann nicht im Array enthalten ist.

Exponentielle Suche im sortierten (unbeschränkten) *Array*

```
1 // A        zu durchsuchendes, 1-indiziertes, aufsteigend sortiertes Array
2 // needle   der Wert, der gesucht wird
3 function ExponentialSearch(A,needle)
4     i := 1
5     while i ⩽ length(A) and A[i] < needle do
6         i := i * 2
7     return BinarySearch(A, floor(i/2) + 1, min(i, length(A)), needle)
```

Die Idee ist erst mit einer exponentiellen Schrittweite zu springen, um dann mit einer binären Suche den Wert zu finden. Die Laufzeit ist $O(\log(i))$ wobei i die Position des gesuchten Wertes ist. Die Laufzeit ist also $O(\log(n))$.

✎ Übung

1.3. Wer sucht, der findet 5?

Folgende Werte sind vom Array A bekannt:

$$A[1] = -27,\ A[15] = 13,\ A[29] = 29$$

Gesucht wird der potentielle Index des Wertes 5. Welcher Index i sollte als nächstes untersucht werden bei binärer, linearer oder quadratisch interpolierender Suche?

1.4. Wer sucht, der findet -1?

Folgende Werte sind vom Array A bekannt:

$$A[1] = -13,\ A[7] = -4,\ A[13] = 11$$

Gesucht wird der potentielle Index des Wertes -1. Welcher Index i sollte als nächstes untersucht werden bei binärer, linearer oder quadratisch interpolierender Suche?

✎ Übung

1.5. Lineare Interpolierende Suche

Setzen Sie den Algorithmus für die lineare interpolierende Suche in einer Programmiersprache Ihrer Wahl um.

Testen Sie den Algorithmus mit folgenden Arrays:

```
A = [1, 3, 5, 7, 9, 11, 13, 15]        # linear verteilt (2x-1)

B = [0, 7, 13, 22, 27, 32, 44, 49]     # approx. 7x linear verteilt

C = [0, 2, 16, 54, 128, 250, 432, 686] # quadratisch verteilt (4x^2)
```

Wie viele Schritte (im Sinne von Schleifendurchläufen) sind maximal notwendig, um festzustellen ob ein Wert im Array enthalten ist oder nicht?

✎ Übung

1.6. Exponentiell Interpolierende Suche

1. Implementieren Sie den Algorithmus für die exponentiell interpolierende Suche in einer Programmiersprache Ihrer Wahl. (Ggf. müssen Sie noch die passende binäre Suche implementieren). Implementieren Sie die Suche basierend auf einer Funktion und nicht über einem Array. Die Funktion kann zum Beispiel eine mathematische Funktion sein, oder eine einfache Funktion die auf einem Array operiert.
2. Gegeben sei die folgende Funktion, die als Generator fungiert. (x sei eine natürliche Zahl).

$$f(x) = \text{round}\left(\frac{1}{1+e^{-x}} \cdot 100\,000\,000\right)$$

   Testen Sie ob 99999996 ein Wert der Funktion ist und geben Sie den Index (x) zurück.
3. Wann macht es Sinn die exponentiell interpolierende Suche zu verwenden?

Hinweise zur Implementierung

In Java kann die zu übergebene Funktion den Typ `java.util.function.DoubleUnaryOperator` haben. Beim Aufruf kann man dann zum Beispiel eine entsprechende Lambda-Funktion angeben werden, die für einen double Wert einen anderen double Wert zurückgibt.

Es kann hilfreich sein alle statischen Methoden und Konstanten der `Math`-Klasse über einen static import zu verwenden.

Die exemplarische Verwendung ist wie folgt:

```
void main() {
    try {
        IO.println(
            "Wert 64 hat Index: " + binarySearch((double x) -> 4 * x + 3, 0, 100, 64)+ "."
        );
    } catch (NoSuchElementException e) {
        IO.println("Wert 64 nicht gefunden.");
    }
    IO.println(binarySearch((double x) -> (4 * x + 3), 0, 100, 43));

    try {
        IO.println(
            "Wert 99999996 hat Index: " +
                exponentialSearch((double x) -> TODO, 99999996 )
        );
    } catch (NoSuchElementException e) {
        IO.println("Wert 99999996  nicht gefunden.");
    }
}
```

2. Selbstanordnende Arrays

Suchen auf Arrays mit spezieller Ordnung

- Sind die Daten nominal skaliert, oder sagt die Ordnung der Werte im Array nichts über die Zugriffshäufigkeit aus, so können Arrays auf Basis der Zugriffe sortiert werden.
- Erfordert prinzipiell eine lineare Suche, die es gilt soweit möglich zu beschleunigen.
- Anwendung(-sgebiete):
  - Cache-Zugriffe, Verwaltung von virtuellem Speicher
  - Wenn Werte häufiger verlangt werden als andere, so besitzen die Anfragen eine Wahrscheinlichkeitsverteilung.
  - Die Verteilung wird durch Abzählen angenähert, da sie nicht bekannt ist. Darauf basierend werden die Werte entsprechend sortiert.

Strategien zur Anordnung

Definition: FC-Regel

Ein Array A ist gemäß *frequency count* (FC-Regel) sortiert, wenn für alle Werte gilt, dass $c(A[k]) \geq c(A[j])$ wenn $k < j$ und $c(x)$ die realisierte Häufigkeit des Wertes x darstellt.

Hinweis

Es wird typischerweise lokal getauscht, um die Ordnung herzustellen.

Definition: MF-Regel

Ein Array A ist gemäß *move to front* (MF-Regel) sortiert, wenn bei Auftritt eines Wertes $A[k]$ in der Folge mit der ersten Position $A[1]$ oder $A[0]$ vertauscht wird, sollte der Wert noch nicht an der ersten Stelle stehen.

Definition: T-Regel

Ein Array A ist gemäß *transpose* (T-Regel) sortiert, wenn bei Auftritt eines Wertes $A[k]$ in der Folge mit der Position davor $A[k-1]$ vertauscht wird, sollte der Wert noch nicht an der ersten Stelle stehen.

Strategien zur Anordnung - Diskussion

- Die FC-Regel erfordert das Mitführen der Häufigkeit der Werte. Die MF-Regel und die T-Regel sind einfacher zu implementieren, da sie nur die Reihenfolge der Werte im Array verändern.
- Für MF-Regel und T-Regel gibt es worst-case Aufrufsequenzen, die immer zu den schlechtesten Laufzeiten führen.
- Die MF-Regel nimmt eher starke Änderungen vor und reagiert schnell.
- Die T-Regel nimmt eher schwache Änderungen vor und ist stabiler.

Zusammenfassung

Die Bewertung sollte anhand der tatsächlichen Daten erfolgen:

- Liegen Häufigkeitsinformationen vor, so ist die FC-Regel sinnvoll.
- Die MF-Regel ist für sich ändernde Verteilungen sinnvoller, die T-Regel für stabilere Situationen.

✎ Übung

2.1. A = [1,2,3,4,5] selbstanordnend sortieren

Das Array `A = [1,2,3,4,5]` soll selbstanordnend sortiert werden. Die gesuchten Werte sind: `1,2,3,2,3,2,1,5`. Bestimmen Sie die Anordnung des Arrays nach jedem Zugriff für die Sortierungen nach MF-Regel, T-Regel und FC-Regel. Füllen Sie die nachfolgende Tabelle aus:

x	MF-Regel	T-Regel	FC-Regel	Häufigkeiten pro Wert
1				
2				
3				
2				
3				
2				
1				
5				

✎ Übung

2.2. A = [1,2,3,4,5,6] selbstanordnend sortieren

Das Array `A = [1,2,3,4,5,6]` soll selbstanordnend sortiert werden. Danach werden die folgenden Werte in der angegebenen Reihenfolge gesucht: `5,1,6,2,3,6,5`. Bestimmen Sie die Anordnung des Arrays nach jedem Zugriff für die Sortierungen nach MF-Regel, T-Regel und FC-Regel. Füllen Sie die nachfolgende Tabelle aus:

x	MF-Regel	T-Regel	FC-Regel	Häufigkeiten
5				
1				
6				
2				
3				
6				
5				

3. Suche nach dem n-ten Element

Suche nach dem n-ten Element - Einführung

- Ist das Array sortiert, so ist die Suche nach dem n-ten Element trivial und hat eine Laufzeit von $O(1)$.
- Ist das Array nicht sortiert, so ist die Suche nach dem n-ten Element nicht trivial.

  Wir unterscheiden:

  1. wird das Array (im Folgenden) auch noch sortiert gebraucht, so ist es am effizientesten dieses erst zu sortieren, um dann das n-te Element auszulesen. Die Laufzeit beträgt dann - mit der Wahl eines geeigneten Sortierverfahrens - $O(n \log n)$.
  2. Ist eine Sortierung nicht erforderlich/gewünscht, so können wir mit Hilfe von Teile-und-Herrsche-Verfahren das n-te Element auch effizienter bestimmen.

Suche nach dem k-ten Element mittels Quickselect

```
// A    ein 0-indiziertes Array
// k    das k-größte Element (0-indiziert)
function Quickselect(A,k)
    if length(A) == 1 then return A[0]
    pivot   := A[length(A)-1] // ein bel. Element als Pivot (hier das letzte)
    lows    := []             // Elemente kleiner als Pivot
    highs   := []             // Elemente größer als Pivot
    pivotsCount := 0          // Anzahl der Pivot-Elemente
    for x in A do             // Partitionierung ...
        if x < pivot then lows.append(x)
        else if x > pivot then highs.append(x)
        else pivotsCount := pivotsCount + 1

    if k < length(lows) then
        return Quickselect(lows, k)
    else if k < length(lows) + pivotsCount then
        return pivot          // das k-te Element ist ein Pivot-Element
    else
        return Quickselect(highs, k - length(lows) -  pivotsCount)
```

※ Hinweis

In einer realen Implementierung sollte das Pivot-Element zufällig gewählt werden, um - für den Fall, dass das Array sortiert ist - die Laufzeit zu verbessern.

※ Hinweis

Der Quickselect Algorithmus kann auch *in-place* implementiert werden, d. h. ohne zusätzlichen Speicherbedarf. Dies setzt voraus, das die ursprüngliche Reihenfolge der Elemente nicht erhalten bleiben muss.

Beispielanwendung: Bestimmung des Medians

```
// A    ein nicht sortiertes, 0-indiziertes Array
function FindMedian(A)
    n := length(A)
    if n % 2 == 1 then // d. h. wir haben eine ungerade Anzahl von Elementen in A
        return Quickselect(A, floor(n / 2))
    else // gerade Anzahl von Elementen in A
        left    := Quickselect(A, floor(n / 2) - 1)
        right   := Quickselect(A, floor(n / 2))
        return (left + right) / 2.0
```

✎ Übung

3.1. n-te Element bestimmen

1. Bestimmen Sie den Median für das Array `A = [23, 335, 2, 24, 566, 3, 233, 54, 42, 6, 667, 7, 5, 7, 7]`. Wenden Sie dazu den Algorithmus `FindMedian` (inkl. `Quickselect-Algorithmus`) an.
2. Geben Sie weiterhin nach jeder Partitionierung im Quickselect Algorithmus den aktuellen Zustand an (d. h. nach Zeile 11 in Quickselect).

Array A	k	Lows	Pivot	Pivots Count	Highs
[...]	<K>	[...]	<P>	<#P>	[...]

✎ Übung

3.2. Komplexität von Quickselect

Bestimmen Sie die Komplexität des Quickselect-Algorithmus im schlechtesten Fall, im Durchschnittsfall und im besten Fall.

Komplexitätsanalyse

Zur Bestimmung der Komplexität kann man entweder das Master Theorem anwenden oder die Anzahl der Schritte für die Partitionierung bestimmen und die Summe der Schritte aufstellen.

Geometrische Reihen

Die Summenformel für eine geometrische Reihe ($S_n = a + ar + ar^2 + \cdots + ar^{n-1}$) lautet:

$$S_n = a \cdot \frac{1 - r^n}{1 - r} \quad \text{für} \quad r \neq 1$$

Mit:

S_n:	Summe der ersten n Glieder der geometrischen Reihe.
a:	Das erste Glied der Reihe.
r:	Der Quotient (Verhältnis aufeinanderfolgender Glieder).
n:	Die Anzahl der Glieder.

Für n gegen unendlich und $|r| < 1$ gilt somit:

$$S = \frac{a}{1 - r} \quad \text{für} \quad |r| < 1$$